

# 操作系统内核技术文档

wllos

feat/ltp

华南师范大学

吴秉谦 李骏东 罗锦程

**2026 年 6 月 26 日**

# 目录

|       |                           |    |
|-------|---------------------------|----|
| 第 1 章 | 摘要 .....                  | 1  |
| 1.1   | 项目定位 .....                | 1  |
| 1.2   | 支持能力 .....                | 1  |
| 第 2 章 | 系统概述 .....                | 2  |
| 2.1   | 总体架构 .....                | 2  |
| 2.2   | 设计思路与实现重点 .....           | 2  |
| 2.3   | 代码结构 .....                | 3  |
| 2.4   | 启动与初始化流程 .....            | 4  |
| 2.5   | 运行与评测路径 .....             | 4  |
| 2.6   | 模块协同关系 .....              | 5  |
| 第 3 章 | 内存管理模块 .....              | 6  |
| 3.1   | 设计目标 .....                | 6  |
| 3.2   | 地址空间结构 .....              | 6  |
| 3.3   | 映射与权限维护 .....             | 7  |
| 3.4   | brk、mmap 与共享内存 .....      | 8  |
| 3.5   | ELF 装载与用户栈 .....          | 8  |
| 3.6   | 写时复制与缺页路径 .....           | 9  |
| 3.7   | 用户内存访问支撑 .....            | 10 |
| 第 4 章 | 进程与调度模块 .....             | 11 |
| 4.1   | 任务模型 .....                | 11 |
| 4.2   | clone 与 execve 生命周期 ..... | 11 |
| 4.3   | 等待队列与阻塞语义 .....           | 12 |
| 4.4   | 退出、等待与父子关系 .....          | 13 |
| 4.5   | 凭据与组管理 .....              | 14 |
| 4.6   | 前台任务与评测调度 .....           | 14 |
| 第 5 章 | 文件系统与 I/O 模块 .....        | 16 |
| 5.1   | VFS 与 fd 表 .....          | 16 |
| 5.2   | 内存文件与大文件策略 .....          | 16 |
| 5.3   | 权限、符号链接与路径解析 .....        | 17 |
| 5.4   | 管道、poll 与非阻塞 I/O .....    | 17 |
| 5.5   | ext4 与 MemFS 协作 .....     | 17 |
| 5.6   | 向量 I/O 与固定偏移 I/O .....    | 18 |
| 5.7   | 文件描述符生命周期 .....           | 18 |
| 第 6 章 | 系统调用与兼容模块 .....           | 20 |
| 6.1   | 系统调用分发 .....              | 20 |
| 6.2   | 用户内存访问 .....              | 20 |
| 6.3   | 信号、futex 与时间接口 .....      | 21 |

|        |                    |    |
|--------|--------------------|----|
| 6.4    | 兼容性状态表 .....       | 21 |
| 6.5    | errno 策略 .....     | 22 |
| 6.6    | 跨架构兼容细节 .....      | 22 |
| 6.7    | 评测导向的 ABI 覆盖 ..... | 23 |
| 第 7 章  | LTP 与评测模块 .....    | 24 |
| 7.1    | harness 设计 .....   | 24 |
| 7.2    | 测试集合 .....         | 25 |
| 7.3    | 有界 LTP 选择 .....    | 25 |
| 7.4    | parser 分组协议 .....  | 25 |
| 7.5    | 评测证据闭环 .....       | 26 |
| 7.6    | 新增 case 准入标准 ..... | 26 |
| 第 8 章  | 测试结果与分析 .....      | 28 |
| 8.1    | 编译与运行方式 .....      | 28 |
| 8.2    | 当前能力总结 .....       | 28 |
| 8.3    | 开发问题与解决方法 .....    | 29 |
| 8.4    | 回归证据记录格式 .....     | 30 |
| 第 9 章  | 总结与展望 .....        | 31 |
| 9.1    | 总结 .....           | 31 |
| 9.2    | 后续工作 .....         | 31 |
| 第 10 章 | AI 辅助使用说明 .....    | 32 |
| 10.1   | 使用原则 .....         | 32 |
| 10.2   | 使用场景与验证方式 .....    | 32 |
| 10.3   | 工程边界与责任 .....      | 32 |

## 第 1 章 摘要

### 1.1 项目定位

wll0s 是一个面向操作系统内核竞赛与课程实践的 Rust 内核项目。`feat/ltp` 分支围绕真实用户程序、真实 libc、真实文件系统镜像和真实评测日志构建 Linux ABI 支持能力，并以模块化方式推进内核功能。本文档按照 UndefinedOS 文档的写法收束为摘要、系统概述、核心模块、测试结果和总结；模块内部重点呈现设计动机、数据结构、关键流程和代表性代码。

该分支的核心价值在于“可证明”。构建阶段通过 `Makefile` 控制 RISC-V64 与 LoongArch64 双架构，运行阶段通过 QEMU 串口日志暴露真实输出，评测阶段通过 `testdata` 中的 `judge` 脚本解析 `basic`、`busybox`、`lua`、`iozone`、`libc-test`、`libcbench`、`lmbench` 与 bounded LTP 的结果。内核以真实用户态输出作为评测依据，并用一组结构清晰的系统调用与模块实现支撑当前测试面。

从工程形态看，wll0s 已经形成了较清晰的内核层次：启动代码负责架构入口和页表切换，内存模块负责地址空间和 VMA，任务模块负责任务生命周期和调度，文件系统模块负责 VFS、fd 与 ext4/MemFS 后端，系统调用模块负责 Linux ABI 入口，`harness` 负责在用户态测试集合和内核任务之间建立可重复的运行路径。本文重点描述这些模块之间如何协作，以及它们如何服务于 `feat/ltp` 分支的真实评测目标。

### 1.2 支持能力

当前分支面向操作系统竞赛评测场景，已经支持真实 ELF 加载、动态解释器路径、用户栈构造、fork/clone/exec/wait 生命周期、VFS 路径操作、文件描述符、管道、socket 兼容入口、signal/futex/time 基础语义，以及 bounded LTP case。上述能力共同支撑 glibc/musl 用户程序、BusyBox 脚本、benchmark 程序和 LTP 子集在 QEMU 环境中的真实运行。

后续扩展将继续围绕线程语义、网络协议栈、文件系统同步、权限模型和更大规模 LTP 用例展开。当前实现已经为这些方向建立了统一的任务对象、地址空间对象、VFS 对象和系统调用分发接口，具备持续扩展的工程基础。

| 模块         | 功能                                                             |
|------------|----------------------------------------------------------------|
| 任务系统       | 维护任务、线程组、等待队列、信号状态和调度入口。                                       |
| 内存管理       | 提供页表、VMA、 <code>mmap</code> 、 <code>brk</code> 、SysV shm 和用户栈。 |
| 文件系统       | 组合 MemFS、ext4 运行时根盘、VFS、fd、pipe 和伪设备。                          |
| 系统调用       | 按 Linux ABI 分发进程、文件、内存、信号、时间和网络相关请求。                           |
| 评测 Harness | 从运行时文件系统发现脚本，运行真实测试并保留串口证据。                                    |

## 第 2 章 系统概述

### 2.1 总体架构

wll0s 的总体结构可以分为五层。最底层是架构相关入口，包含 RISC-V64 与 LoongArch64 的启动汇编、链接脚本、SBI 或平台抽象相关初始化。其上为基础内核服务，包括堆分配器、物理页分配器、页表封装、定时器、日志和控制台。再往上是核心运行时，包括任务、地址空间、文件系统、trap 和系统调用。最上层是用户程序运行与评测 harness，它负责发现镜像中的脚本和二进制，按照 libc 根目录组织测试，并把串口输出整理成 judge 可识别的分组。

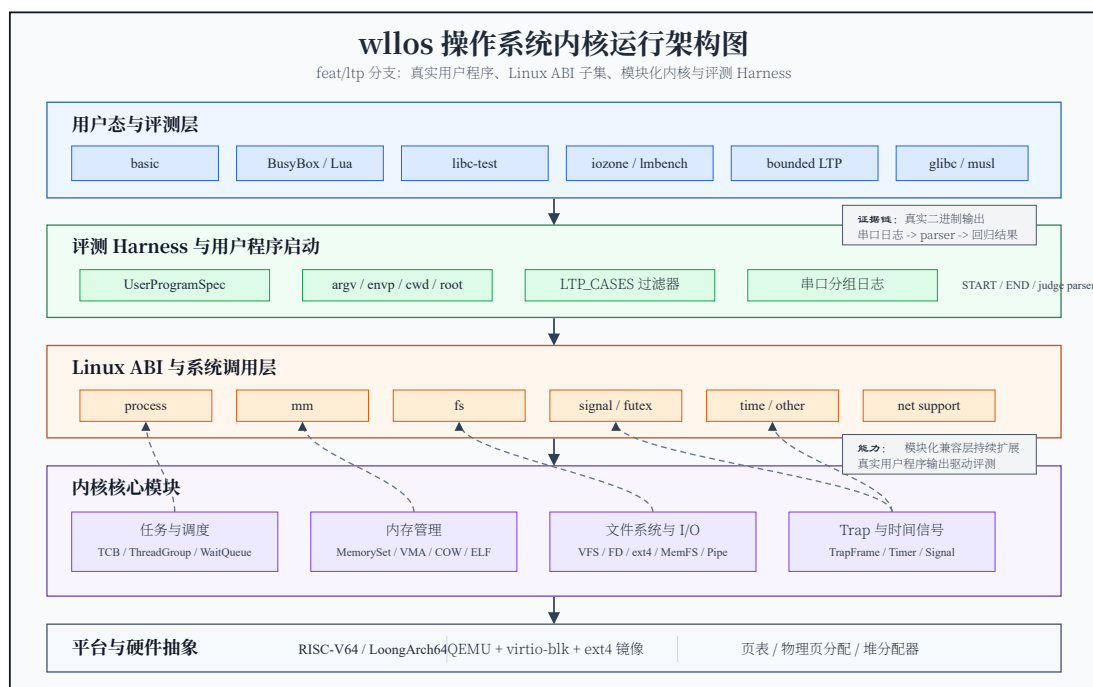


图 2-1 系统总体架构

这套结构的优点是职责划分清楚。用户程序由 `UserProgramSpec` 描述 `argv`、`envp`、`cwd`、`root` 和 `marker`；系统调用层通过 `fd/VFS` 后端统一读写；任务调度用 `wait queue` 统一描述 `IO`、`child exit`、`timer`、`futex` 和 `signal` 的等待原因。模块之间的协作集中在少数生命周期函数中，例如 `execve` 会同时衔接 `ELF`、`MemorySet`、`fd close-on-exec`、`signal action` 和 `trap frame`，便于开发者定位启动链路。

### 2.2 设计思路与实现重点

wll0s 的开发思路是以真实用户程序作为牵引，以稳定内核对象作为支点，再用可复现评测结果验证每一轮实现。内核围绕“程序能否从镜像读取、能否完成动态链接、能否建立地址空间、能否通过 `fd` 与 `VFS` 交互、能否被父进程等待回收”这条主线组织模块。这样

的路线使进程、内存、文件系统和系统调用始终服务于同一个目标：让真实二进制以接近 Linux 用户态的方式运行。

实现重点可以概括为四个方面。第一，核心对象要稳定，TaskControlBlock、MemorySet、fd table 和 VFS node 都承载清晰生命周期。第二，ABI 细节要集中处理，用户指针复制、errno、trap frame、auxv 和 clone flag 收敛到统一辅助路径中。第三，评测路径要可追溯，harness、串口日志和 judge parser 形成闭环。第四，双架构支持要共用语义层，把寄存器、TLS 和入口差异收敛在架构适配位置。

| 技术思路     | 实现重点                                       | 评测价值                         |
|----------|--------------------------------------------|------------------------------|
| 真实程序驱动   | 优先打通 ELF、动态链接器、libc、shell 和测试二进制。          | 评测结论由真实用户态行为给出，并与内核内部检查相互印证。 |
| 对象模型先行   | 用统一任务、地址空间、fd、VFS 和等待队列承载语义。               | 复杂 case 可以拆回具体内核对对象，便于定位与回归。 |
| ABI 语义收敛 | 集中处理用户指针、errno、auxv、clone flags 和架构参数差异。   | 减少重复实现，使 LTP 失败路径也有稳定解释。     |
| 模块协同推进   | 按进程、内存、文件、信号、时间等子系统组织实现节奏。                 | 每组新增 case 都对应明确模块能力。         |
| 证据链回归    | 保存构建命令、串口日志、parser summary 和 focused case。 | 评委和开发者可以复现同一条运行链路。           |

## 2.3 代码结构

项目根目录中的 ltp/ 是从 feat/ltp 分支拉取的源码。它同时保存了内核代码、构建脚本、评测脚本、文档和离线依赖。当前技术文档不复制整个文件树，只列出对理解内核最重要的目录。

| 路径             | 作用                                                                  |
|----------------|---------------------------------------------------------------------|
| os/src/main.rs | 内核初始化入口，完成日志、内存、设备、文件系统、任务系统与 harness 启动。                           |
| os/src/mm      | 内存管理模块，包含页表、物理页分配、VMA、ELF 加载、COW 和映射维护。                             |
| os/src/task    | 任务管理模块，包含任务控制块、调度器、等待队列、pid、上下文和评测 harness。                         |
| os/src/fs      | 文件系统模块，包含 fd 表、VFS、MemFS、ext4 后端、pipe、socket 与伪设备。                  |
| os/src/syscall | 系统调用模块，按照 Linux syscall id 分发到 process、fs、mm、signal、other、net 等子模块。 |
| os/src/trap    | 异常和系统调用 trap 处理，保存当前 syscall 上下文，处理 execve 后的返回路径。                  |
| scripts        | 基线运行、评测汇总、镜像修复和测试套件补丁脚本。                                            |

| 路径                    | 作用                                        |
|-----------------------|-------------------------------------------|
| <code>testdata</code> | <code>judge</code> 脚本与配置，用于从串口日志中提取测试组结果。 |

## 2.4 启动与初始化流程

内核启动流程可以理解为从“平台可运行”逐步过渡到“用户程序可运行”。早期入口负责建立栈和基本地址空间，随后初始化日志、堆、物理页、文件系统和任务系统。块设备和运行时根目录准备完成后，`harness` 启动用户程序。该顺序体现了运行依赖关系：文件系统提供动态解释器和测试脚本，任务系统提供调度入口，`timer` 为 `sleep`、`poll`、`futex timeout` 和 `interval timer` 提供时间基础。

| 阶段      | 主要对象                                                                              | 说明                                                      |
|---------|-----------------------------------------------------------------------------------|---------------------------------------------------------|
| 早期入口    | 链接脚本、启动栈、 <code>trap</code>                                                       | 完成架构相关最低限度初始化，使 Rust 内核代码可以运行。                          |
| 内存初始化   | <code>heap</code> 、 <code>frame allocator</code> 、 <code>kernel page table</code> | 为后续动态对象、页表映射和用户地址空间分配提供基础。                              |
| 设备与文件系统 | <code>virtio-blk</code> 、 <code>ext4</code> 、 <code>MemFS</code>                  | 建立运行时根目录，使用户程序、动态库和测试脚本可被读取。                            |
| 任务系统    | <code>processor</code> 、 <code>task manager</code> 、 <code>wait queue</code>      | 创建 <code>harness</code> 或 <code>init</code> 任务，并准备调度循环。 |
| 用户态运行   | <code>ELF loader</code> 、 <code>syscall</code> 、 <code>trap frame</code>          | 进入真实测试程序，由系统调用和异常路径支撑后续执行。                              |

```
pub fn rust_main() -> ! {
    logging::init();           // serial log and judge evidence
    mm::init();                // heap and frame allocator
    trap::init();              // exception and syscall entry
    drivers::init();           // platform devices
    fs::init();                 // ext4, MemFS and pseudo devices
    task::init_harness();       // foreground test harness
    task::run_tasks();          // scheduler loop
}
```

代码 2-1 内核启动主线示意

这段示意用于概括启动依赖关系。`wll0s` 的测试路径高度依赖串口输出，因此日志初始化较早完成；用户程序依赖 `ext4` 镜像中的动态库，因此文件系统初始化先于 `harness`；任务调度依赖 `trap frame` 和地址空间激活，因此内存和 `trap` 初始化位于更前阶段。构建脚本、内核初始化和评测脚本共同构成了完整运行链路。

## 2.5 运行与评测路径

`Makefile` 是 `feat/ltp` 分支的运行主线。它默认以 RISC-V64 为主架构，同时保留 LoongArch64 构建路径；通过 `LTP=1` 打开 LTP feature，通过 `LTP_CASES` 指定有界 case 集。与普通课程内核只运行一个 `init` 程序不同，这个分支的内核启动后会进入 `harness`，由 `harness` 按照镜像内容依次运行不同测试组。这样做的价值是所有测试都通过真实用

户程序触发内核语义，也要求内核在 ABI、errno、路径解析和任务生命周期上形成更完整的工程闭环。

```
LTP ?= 0
FOCUSED_LTP_CASES ?= writev01,setegid02,getgroups01,setgroups01,\
setgroups02,setgroups03,setgroups04,access01,open02,setfsuid01,\
setfsgid01,faccessat01,access02,open03,symlink01,readlink01,\
lstat01,lstat02,symlink02,symlink03,symlink04,symlinkat01,\
readlinkat01,setitimer02,setitimer01,getitimer01,getitimer02

ifeq ($(LTP),1)
    LTP_EXTRA := --features ltp
else
    LTP_EXTRA :=
endif
```

代码 2-2 构建脚本中的 LTP 控制入口

这段代码体现了当前 LTP 策略：通过本地 `all` 或显式 `LTP=1` 路径打开聚焦集合。`FOCUSED_LTP_CASES` 覆盖身份、组、访问权限、符号链接、`readlink/lstat` 和 `interval timer` 等主题，正好对应本分支近期补强的系统调用语义。后续扩展 LTP 时，将继续按子系统逐步增加 `case`，使每一组评测都对应明确的内核能力。

## 2.6 模块协同关系

wllos 的模块化并不意味着各模块互相孤立。真实用户程序运行时，系统调用会把进程、内存、文件系统、信号和时间模块串联起来。例如 `execve` 需要 VFS 读取可执行文件，ELF loader 构建新地址空间，任务模块更新 `exec_path` 和 `trap frame`，`fd` 表关闭 `FD_CLOEXEC` 描述符，`signal` 模块重置信号动作，最终由 `trap` 层返回用户入口。这样的协同路径体现了内核对象设计的统一性。

评测程序越接近真实 Linux 用户态，模块协同越重要。`BusyBox shell` 依赖 `fork/exec/wait`、`pipe`、路径解析和脚本解释；`iozone` 依赖文件系统、SysV `shm`、固定偏移 I/O 和进程等待；`bounded LTP` 则会刻意组合身份、权限、路径、用户指针和 `errno`。wllos 将这些能力收敛在稳定的核心对象上，使后续扩展可以沿模块接口推进。

| 用户态行为                     | 参与模块                               | 协作说明                                                 |
|---------------------------|------------------------------------|------------------------------------------------------|
| <code>execve</code> 启动程序  | VFS、ELF loader、MemorySet、Task、Trap | 读取文件、装载段、构建用户栈并切换入口。                                 |
| <code>fork + wait4</code> | Task、MemorySet、WaitQueue、Signal    | 复制任务上下文、COW 地址空间并通知父进程。                              |
| Shell 管道                  | FD、Pipe、WaitQueue、Signal           | 维护 <code>pipe</code> 缓冲、阻塞唤醒和 <code>SIGPIPE</code> 。 |
| LTP 权限 case               | Credentials、VFS、Syscall user copy  | 按身份检查路径访问并返回可解释 <code>errno</code> 。                 |
| Benchmark I/O             | FD、ext4、MemFS、SysV shm、Timer       | 支撑多进程文件吞吐、共享内存和计时。                                   |



## 第 3 章 内存管理模块

### 3.1 设计目标

内存管理模块需要同时满足三个目标。第一，能够装载真实 ELF，包括动态链接程序、解释器、用户栈、auxv 和 signal trampoline。第二，能够支持 libc 和测试程序常见的 `brk`、`mmap`、`munmap`、`mprotect` 与 SysV shared memory。第三，能够支撑 `clone`、`fork` 和 `execve` 的地址空间生命周期，尤其是 `fork` 后的写时复制和 `exec` 后的地址空间替换。

当前实现的主对象是 `MemorySet`。它持有一个根页表和一组 `MapArea`，每个 `area` 描述一段用户虚拟地址范围、权限和 `backing` 类型。匿名页、文件映射页、共享内存页都通过 `area` 进入统一维护路径。页表负责硬件可见映射，`area` 则是内核理解用户地址空间语义的来源。这个分离很关键：`mprotect`、`munmap` 和 `fork` COW 不应只改页表，还要同步修改 VMA 元数据，否则后续 `lazy fault`、用户拷贝或 `exec` 清理都会失去依据。

### 3.2 地址空间结构

```
/// Per-task or shared user address space.
/// page_table is the hardware-visible root page table.
/// areas keeps VMAs for mmap, munmap, mprotect and fork.
pub struct MemorySet {
    pub page_table: PageTableWrapper,
    pub areas: Vec<MapArea>,
}

impl MemorySet {
    /// Create an empty address space for ELF load or execve.
    pub fn new_bare() -> Self {
        Self {
            page_table: PageTableWrapper::alloc_new(),
            areas: Vec::new(),
        }
    }
}
```

代码 3-1 `MemorySet` 的核心结构

在文档层面看，这段结构很小，但它贯穿整个系统。任务控制块中的 `memory_set` 使用 `Arc<Mutex<MemorySet>>` 包装，因此 `CLONE_VM` 可以让多个任务共享同一个地址空间；普通 `fork` 则通过 `fork_cow` 生成新的 `MemorySet`，并把可写私有页转换成 COW 权限；`execve` 会构造全新的 `MemorySet`，再替换当前任务的地址空间。这个选择使内核能用相对统一的对象模型覆盖进程、线程和测试 `harness` 的不同路径。

### 3.3 映射与权限维护

内存映射的实现重点在于同时维护 VMA 语义和页表状态。`insert_framed_area_with_backing` 会先对虚拟地址做页对齐, 检查区间合法性, 再根据权限决定是否立即分配物理页。对于 `PROT_NONE` 一类保留映射, `area` 可以先进入列表但不建立叶子页表项; 等 `mprotect` 授予读写执行权限时, 再补齐帧并重写页表。这样可以兼容 `libc` 的 TLS、DTV 扩展以及部分测试中先保留再启用的映射模式。

```
pub fn insert_framed_area_with_backing(
    &mut self,
    start_va: VirtAddr,
    end_va: VirtAddr,
    permission: PTEFlags,
    backing: MapAreaBacking,
) -> Result<(), SysErrNo> {
    let start = align_down(start_va.raw());
    let end = align_up(end_va.raw()).ok_or(SysErrNo::EINVAL)?;
    if start >= end {
        return Err(SysErrNo::EINVAL);
    }

    let mut area = MapArea::with_backing(
        VirtAddr::new(start),
        VirtAddr::new(end),
        permission,
        backing,
    );

    // PROT_NONE-style ranges record the VMA first.
    if !has_leaf_permission(permission) {
        self.areas.push(area);
        self.coalesce_areas();
        return Ok(());
    }

    // Leaf mappings allocate frames and install PTEs.
    for vpn in start / PAGE_SIZE .. end / PAGE_SIZE {
        let frame = frame_allocator::alloc_frame().ok_or(SysErrNo::ENOMEM)?;
        let vaddr = VirtAddr::new(vpn * PAGE_SIZE);
        let paddr = PhysAddr::new(frame.ppn().addr());
        self.page_table.map_page(
            vaddr,
            paddr,
            effective_mapping_flags(permission),
            MappingSize::Page4KB,
        );
        area.frames.push(frame);
    }

    self.areas.push(area);
    self.coalesce_areas();
}
```

```
Ok())
}
```

代码 3-2 带 backing 的匿名或文件映射

这段摘录保留了实现的关键思想：VMA 和页表同时维护，失败时返回真实 `errno`。这里的 `effective_mapping_flags` 还会处理 COW 页，将逻辑上可写但当前处于写时复制状态的页临时移除写权限，以便后续写异常触发复制。对 RISC-V 来说，`PROT_WRITE` 还需要配合读权限，因为硬件 PTE 要求写权限伴随读权限，这些架构差异被收敛在权限转换函数中。

### 3.4 brk、mmap 与共享内存

`brk` 主要服务 libc 分配器。当前实现维护逻辑 break、已映射 break 和 heap 起点：`brk(0)` 返回当前 break，合法增长会补充匿名页，合法收缩会释放上方页，不合法请求则保持旧 break。这种行为符合 Linux raw `brk` 的兼容预期，比简单返回错误更适合 libc。

`mmap` 覆盖匿名映射、文件映射和共享内存。匿名映射用于栈、堆外分配、动态链接器临时区域和 pthread 栈；文件映射用于动态库或测试程序读入文件内容；SysV shared memory 用于 iotest 和部分 libc 测试。共享可写文件映射会在 `munmap` 或固定替换时写回，后续可继续结合 dirty tracking 和 `msync` 进一步增强持久化语义。

| 接口                        | 完成情况 | 说明                              |
|---------------------------|------|---------------------------------|
| <code>brk</code>          | 已实现  | 维护真实堆区增长和收缩，非法请求保留旧 break。      |
| <code>mmap</code>         | 已支持  | 覆盖匿名映射、文件映射和常见 libc 映射路径。       |
| <code>munmap</code>       | 已支持  | 支持区间拆分、释放和共享文件映射写回。             |
| <code>mprotect</code>     | 已支持  | 支持权限拆分、页表重映射和保留 VMA 激活。         |
| <code>shmget/shmat</code> | 已支持  | 提供真实共享物理页，覆盖 iotest 和基础 IPC 场景。 |

### 3.5 ELF 装载与用户栈

真实用户程序启动依赖 ELF loader。wll0s 读取 ELF header 和 program header，把 `PT_LOAD` 段映射到用户地址空间，再处理动态链接程序的解释器路径。对于动态 ELF，内核先装载解释器，并把目标 ELF 的信息通过 `auxv` 传给解释器。glibc 和 musl 对 `auxv`、`argv`、`envp`、栈对齐和 `AT_PHDR/AT_PHNUM/AT_ENTRY` 的要求较高，因此该路径是用户态兼容性的核心。

```
pub fn new_user(elf_data: &[u8]) -> Result<Arc<Self>, SysErrNo> {
    let elf = ElfFile::parse(elf_data)?;
    let (mut memory_set, user_stack_top, entry) = elf.load()?;

    // Build argc, argv, envp and auxv on the user stack.
```

```

let sp = setup_user_stack_for_init(
    &memory_set,
    user_stack_top,
    entry,
    phdr_vaddr,
    phnum,
);

let mut trap_frame = TrapFrame::new();
prepare_user_trapframe(&mut trap_frame);
trap_frame[TrapFrameArgs::SP] = sp; // user stack top
trap_frame[TrapFrameArgs::SEPC] = entry; // user entry
set_user_entry_registers(&mut trap_frame, sp, 1);

// Signal return depends on a user-space trampoline.
signal::install_signal_trampoline(&mut memory_set)?;
/* Create TaskControlBlock and register it. */
Ok(task)
}

```

代码 3-3 创建用户任务时的装载关键路径

这里的栈构造是启动链路中的关键环节。真实 `libc` 会从入口处解析 `argc/argv/envp/auxv`，动态链接器还会根据 `auxv` 定位目标程序头表、页大小、用户身份和随机种子。`feat/ltp` 分支将这些 ABI 细节纳入内存与进程模块的交界处，从而支撑 `glibc/musl` 用户程序稳定启动。

### 3.6 写时复制与缺页路径

`fork` 直接复制所有用户页虽然简单，但在 `iozone`、`libcbench` 和多进程测试中会带来明显成本。`wll0s` 使用 COW 思路处理普通 `fork`：父子地址空间先共享只读物理页，并在页表权限中去掉写权限；当某一方写入时，通过缺页或写异常分配新页、复制内容并恢复写权限。这样既能保持 `fork` 语义，又能避免不必要的全量复制。

当前 COW 与 `MapArea` 的 `backing` 类型、页权限、`frame ownership` 紧密关联。匿名私有页和私有文件映射适合 COW；共享内存和共享文件映射保持共享语义；只读页可以直接共享；保留 VMA 在授予叶子权限后再建立映射。这些规则共同保证 `mmap`、`fork_cow` 和缺页处理保持一致。

| 区域类型                       | fork 行为 | 原因                  |
|----------------------------|---------|---------------------|
| 匿名私有页                      | COW     | 常见堆、栈和私有映射，写入时才复制。  |
| 只读文件页                      | 共享      | 没有写权限，父子共享物理页即可。    |
| 共享内存                       | 共享      | 语义要求多个进程观察同一物理内容。   |
| 共享文件映射                     | 共享/写回   | 需要保持共享语义，当前写回粒度仍较粗。 |
| <code>PROT_NONE</code> 保留区 | 仅复制 VMA | 没有叶子页表项，授权时再分配或装载。  |

### 3.7 用户内存访问支撑

系统调用模块频繁读写用户空间指针，例如路径字符串、`argv/envp`、`iovec`、`stat` 结构、`timespec` 和 `sigset`。wll0s 将用户内存访问封装为统一辅助接口，并在访问前后切换合适的页表上下文。这样做可以把 `EFAULT`、字符串长度、结构体复制和跨页访问处理收敛到一处，减少每个系统调用重复实现用户指针检查的成本。

用户内存访问还与 `lazy` 映射和文件映射有关。部分用户缓冲区可能位于 `mmap` 建立的区域中，内核在读取参数或复制数据时需要确保 `VMA`、页表和物理页状态一致。内存模块提供的 `VMA` 元数据使系统调用能够判断地址范围是否属于用户空间、是否具备读写权限，以及是否需要触发映射补齐。

| 访问对象  | 典型系统调用                                                                            | 内存模块作用                                                       |
|-------|-----------------------------------------------------------------------------------|--------------------------------------------------------------|
| C 字符串 | <code>openat</code> 、 <code>execve</code> 、 <code>readlinkat</code>               | 按页读取字符串，处理终止符和路径长度。                                          |
| 结构体   | <code>statx</code> 、 <code>getitimer</code> 、 <code>sigaction</code>              | 复制固定布局对象并在错误地址返回 <code>errno</code> 。                        |
| 缓冲区   | <code>read</code> 、 <code>write</code> 、 <code>readv</code> 、 <code>writew</code> | 校验用户区间，配合 <code>VMA</code> 和页表完成数据传输。                        |
| 用户栈   | <code>execve</code> 、信号投递                                                         | 写入 <code>argv/envp/auxv</code> 和 <code>signal frame</code> 。 |

## 第 4 章 进程与调度模块

### 4.1 任务模型

wll0s 的任务模型围绕 `TaskControlBlock` 展开。一个任务既可以表示进程中的主线程，也可以表示 `CLONE_THREAD` 创建的线程。为兼容 Linux 语义，任务中同时保留 `pid`、`thread group`、父子关系、`fd` 表、`fs context`、`mm context`、`credentials`、`signal state`、`trap frame`、调度状态和等待状态。该结构将真实 `libc` 和 `LTP` 需要观察的进程状态集中管理，便于 `clone`、`execve`、`exit` 和 `wait4` 等生命周期操作协同完成。

```
pub struct TaskControlBlock {
    pub pid: Pid,                                // TID
    pub thread_group: Arc<ThreadGroup>,         // TGID
    pub inner: Mutex<TaskControlBlockInner>,
    pub memory_set: SharedMemorySet,             // shared by CLONE_VM
    pub fs: SharedFsContext,                     // shared by CLONE_FS
    pub mm: SharedMmContext,                     // brk and next_mmap
    pub credentials: Mutex<Credentials>,        // uid/gid/fsuid/fsgid/groups
    pub signal_state: Mutex<SignalState>,
    pub trap_frame: Mutex<Option<TrapFrame>>,
    pub status: Mutex<TaskStatus>,
    pub block_reason: Mutex<Option<BlockReason>>,
    pub wait_outcome: Mutex<Option<WaitOutcome>>,
}
```

代码 4-1 任务控制块中的关键字段

这里最值得注意的是共享对象的粒度。Linux 的 `clone` 通过 `flags` 精细控制资源共享。wll0s 用 `SharedMemorySet`、`SharedFdTable`、`SharedFsContext` 和 `SharedMmContext` 对应 `CLONE_VM`、`CLONE_FILES`、`CLONE_FS` 等语义；资源独立时则深拷贝或复制上下文。这个模型让 `fork`、`vfork`、线程创建和 `harness` 前台任务都能走同一套生命周期函数。

### 4.2 `clone` 与 `execve` 生命周期

`sys_clone` 是进程模块中最重要系统调用之一。它需要验证 `flag` 组合，复制当前 `trap frame`，让子任务返回值为 0，处理用户栈和 `TLS`，然后根据 `flags` 决定共享或复制地址空间、`fd` 表和 `fs context`。对于不共享地址空间的情况，当前实现使用 `fork_cow`，避免 `fork` 时立即复制所有物理页；对于 `CLONE_VM`，则直接共享父任务的 `MemorySet`。

```
let clone_bits = flags & !CSIGNAL;
if (clone_bits & CLONE_SIGHAND) != 0 && (clone_bits & CLONE_VM) == 0 {
    return Err(SysErrNo::EINVAL);
}
if (clone_bits & CLONE_THREAD) != 0 && (clone_bits & CLONE_SIGHAND) == 0 {
    return Err(SysErrNo::EINVAL);
}
```

```

}

let mut child_tf = crate::trap::clone_current_trapframe()
    .ok_or(SysErrNo::EINVAL)?;
child_tf[TrapFrameArgs::RET] = 0;      // child returns 0
child_tf.syscall_ok();                 // advance child PC

let share_vm = (clone_bits & CLONE_VM) != 0 && !is_vfork;
let memory_set = if share_vm {
    parent.memory_set.clone()
} else {
    let mut parent_memory = parent.memory_set.lock();
    let child_memory = parent_memory.fork_cow()?;
    parent_memory.activate();
    new_shared_memory_set(child_memory)
};

```

代码 4-2 clone 中的资源共享选择

`execve` 则是另一条关键路径。它读取用户路径和参数，解析脚本或 ELF，处理解释器，装载新地址空间，重建用户栈和 `auxv`，关闭 `FD_CLOEXEC` 描述符，重置信号处理，并通知 `trap` 层使用 `exec` 后的返回路径。这里的细节直接影响 `BusyBox`、动态链接器、`libc-test` 和 `LTP` 的稳定性。`/proc/self/exe`、解释器路径、环境变量和 `auxv` 共同决定用户程序能否稳定穿过 `libc` 初始化阶段。

### 4.3 等待队列与阻塞语义

真实测试中的阻塞覆盖 `sleep`、`pipe` 读写、`wait4`、`futex`、`timer`、`signal`、`poll/select` 等多类场景，并且要处理超时、信号中断、任务退出和过期唤醒。`wll0s` 通过 `WaitQueue` 统一这些场景，每个等待项带有 `task`、`token` 和可选 `key`。`token` 用于区分同一个任务不同时间进入的等待，避免一个旧 `timeout` 唤醒新等待。

```

pub fn sleep_until_key_if<F>(
    &self,
    key: Option<WaitKey>,
    deadline_us: Option<usize>,
    should_sleep: F,
) -> Result<WaitOutcome, SysErrNo>
where
    F: FnOnce() -> Result<bool, SysErrNo>,
{
    let task = current_task().ok_or(SysErrNo::ESRCH)?;
    let token = task.next_wait_token();
    *task.wait_outcome.lock() = None;
    *task.block_reason.lock() = Some(self.reason);

    self.waiters.lock().push_back(WaitEntry {
        task: task.clone(),
        token,
        key,
    });
}

```



```

});

// Recheck after enqueue to avoid lost wakeups.
if !should_sleep()? {
    self.remove_waiter(task.pid.0, token);
    *task.block_reason.lock() = None;
    return Ok(WaitOutcome::Woken);
}

if let Some(deadline) = deadline_us {
    crate::timer::add_timeout(deadline, task.clone(), token);
}
block_current_for(self.reason, deadline_us);

*task.block_reason.lock() = None;
let still_waiting = self.remove_waiter(task.pid.0, token);
Ok(finish_wait(&task, still_waiting, deadline_us))
}

```

代码 4-3 等待队列的 token 化睡眠流程

这段代码体现了内核在阻塞语义上的工程取舍。等待队列保证条件检查在入队后再执行一次，信号可中断等待，deadline 由 timer 子系统统一唤醒，任务退出时会清理相关等待项。对于 LTP 来说，errno、超时和中断语义是重要评测内容，因此 wait queue 是进程调度、I/O、futex 和 timer 的共同基础。

## 4.4 退出、等待与父子关系

进程退出路径需要同时处理线程级和进程级状态。普通 `exit` 只终止当前线程；当线程组中的最后一个用户线程退出时，整个进程才变为 zombie。`exit_group` 则要终止同一 TGID 下的所有用户线程，清理 ready queue、futex queue、timer queue、IO wait queue、child wait queue 和 signal wait queue，并向父进程投递 `SIGCHLD`。`wait4` 需要在父进程的子进程集合中查找 zombie，写回退出状态，并在没有可回收子进程但还有活子进程时阻塞。

这个设计比“子进程结束后直接从任务表删除”复杂，但它是 Linux 兼容所必需的。BusyBox shell、libc-test 和 LTP 都会假设父进程能通过 wait 系列调用观察子进程退出状态。如果内核提前释放任务，父进程会得到 `ECHILD`；如果不清理等待队列，已经退出的任务可能被旧 timeout 或旧 wake 再次唤醒；如果不处理 `SIGCHLD`，信号等待类测试会看不到子进程完成。

| 事件                      | 状态变化          | 影响                             |
|-------------------------|---------------|--------------------------------|
| <code>exit</code>       | 当前 TID 退出     | 清理线程私有状态，最后线程退出时标记进程 zombie。   |
| <code>exit_group</code> | 整个 TGID 退出    | 终止同组用户线程，统一唤醒父进程和等待者。          |
| <code>wait4</code>      | 回收 zombie 子进程 | 写回 status，释放父子关系，避免 zombie 泄漏。 |



| 事件       | 状态变化         | 影响                                 |
|----------|--------------|------------------------------------|
| 任务被 kill | 从队列中移除       | 避免 ready/wait/futex/timer 中残留过期任务。 |
| 父进程退出    | 子进程交给 reaper | 防止孤儿进程丢失回收路径。                      |

## 4.5 凭据与组管理

bounded LTP 中的 `setegid02`、`getgroups01` 和 `setgroups01` 到 `setgroups04` 直接验证任务凭据。wll0s 因此为任务保存 real/effective/saved UID/GID、filesystem UID/GID 和 supplementary groups。`clone` 会继承凭据，`execve` 保留当前凭据，`setuid/setgid/setresuid/setresgid` 根据 root-capable 或已有身份集合决定是否允许切换。文件系统权限检查则可以选择 real identity 或 filesystem identity，这正是 `access` 与 `open` 的区别之一。

```
pub struct Credentials {
    pub real_uid: u32,
    pub effective_uid: u32,
    pub saved_uid: u32,
    pub fsuid: u32,
    pub real_gid: u32,
    pub effective_gid: u32,
    pub saved_gid: u32,
    pub fsgid: u32,
    supplementary_groups: Vec<u32>,
}

impl Credentials {
    pub fn is_in_filesystem_group(&self, gid: u32) -> bool {
        self.fsgid == gid
        || self.supplementary_groups.iter().any(|group| *group == gid)
    }
}
```

代码 4-4 任务凭据结构

当前凭据实现围绕 real/effective/saved id、fsuid/fsgid 和 supplementary groups 展开，已经支撑文件访问、组列表和身份切换相关评测。后续可在此基础上扩展 capability、namespace 和更复杂的权限矩阵，使 VFS 权限模型覆盖更广泛的 Linux 场景。

## 4.6 前台任务与评测调度

评测场景中的调度目标与通用服务器不同。wll0s 需要保证测试程序能够按顺序启动、在超时范围内运行、在退出后被 harness 收集结果，并在必要时切换到下一组测试。为此，任务模块保留了前台任务集合、前台 deadline、requeue 逻辑和 child wait queue，使 harness 可以以可控方式运行 basic、busybox、lua、benchmark 与 LTP。

前台运行策略的核心是“真实运行、可控收束”。用户程序仍然通过普通系统调用进入内核，任务退出仍然走 `exit` 或 `exit_group`，父进程仍然通过 `wait` 语义回收子进程。

harness 只负责组织测试顺序、设置环境变量和收集串口输出，不改变用户程序与内核交互的基本路径。

| 调度对象             | 作用           | 评测价值                            |
|------------------|--------------|---------------------------------|
| 前台任务集合           | 记录当前测试相关任务   | 便于 harness 判断测试是否结束。            |
| deadline         | 限制单个前台任务运行时间 | 避免单个 case 长时间占用评测窗口。            |
| child wait queue | 等待子进程退出      | 支撑 shell、BusyBox 和 LTP 的父子进程模型。 |
| requeue 逻辑       | 把仍可运行任务放回队列  | 维持合作式调度下的前台推进能力。                |
| orphan reaper    | 接管孤儿子进程      | 保持进程回收路径稳定。                     |

## 第 5 章 文件系统与 I/O 模块

### 5.1 VFS 与 fd 表

文件系统模块是 `feat/ltp` 分支中扩展最快的部分。它需要同时服务 BusyBox shell、动态链接器、libc 文件访问、iozone 大量读写、LTP 权限与符号链接 case。当前实现把文件对象抽象成 fd 表中的 `FileDescriptor`，再由不同后端提供读写、seek、stat、truncate、目录项、pipe、socket 和伪设备能力。这样的结构让 `openat`、`read`、`write`、`readv`、`writew`、`pread64`、`pwrite64`、`lseek`、`getdents64` 等接口都能通过同一套 fd 路径分派。

| 对象     | 代表路径                                                             | 设计说明                                     |
|--------|------------------------------------------------------------------|------------------------------------------|
| MemFS  | <code>/tmp</code> 、 <code>/var/tmp</code> 、 <code>overlay</code> | 提供可写内存文件和白化覆盖，适合临时文件与测试脚本。               |
| ext4   | <code>/glibc</code> 、 <code>/musl</code> 根目录                     | 承载真实用户程序、动态库、脚本和测试数据。                    |
| 伪设备    | <code>/dev/null</code> 、 <code>/dev/zero</code>                  | 提供字符设备元数据和真实 <code>null/zero</code> 行为。  |
| Pipe   | <code>pipe2</code> 创建                                            | 有界内核缓冲区，支持阻塞与 <code>O_NONBLOCK</code> 。  |
| Socket | <code>socket</code> 创建                                           | 提供本地 loopback 风格队列，已接入 fd 生命周期与 poll 路径。 |

fd 表不仅保存文件对象，还保存 offset、flags、close-on-exec 和访问权限。`FD_CLOEXEC` 对动态链接器、shell 和测试框架很重要，因为 `execve` 后泄漏 fd 会改变程序行为。`O_APPEND`、`O_NONBLOCK`、`F_GETFD/F_SETFD`、`F_SETFL` 也会影响 BusyBox 和 LTP case。当前实现已经支持常见路径，但完整的共享 open-file description 仍然是后续方向。

### 5.2 内存文件与大文件策略

MemFS 的一个细节是大文件内容不总是使用单个 `Vec<u8>`。当文件超过阈值后，内容会转换为 chunked 表示，空洞 chunk 可以用 `None` 表示。这种设计在测试中很有用：iozone、libcbench 或临时文件可能产生较大缓冲，如果每次都复制完整连续数组，内核内存压力会很快上升。chunked 表示牺牲了一点实现复杂度，但换来了更可控的内存增长。

```
pub enum MemFileContent {
    Inline(Vec<u8>),
    Chunked {
        len: usize,
        chunks: Vec<Option<Vec<u8>>>,
    },
}
```

```
impl MemFileContent {
    pub fn from_slice(content: &[u8]) -> Self {
        if content.len() <= MEM_FILE_INLINE_LIMIT {
            return Self::Inline(content.to_vec());
        }

        let mut chunks = Vec::new();
        for part in content.chunks(MEM_FILE_CHUNK_SIZE) {
            if part.iter().all(|byte| *byte == 0) {
                chunks.push(None); // zero chunk is sparse
            } else {
                chunks.push(Some(part.to_vec()));
            }
        }
        Self::Chunked { len: content.len(), chunks }
    }
}
```

代码 5-1 MemFileContent 的分层存储

这种实现能够减少测试场景中的重复零页开销。对文件系统模块来说，最重要的是读写路径保持一致：普通 read/write、固定偏移 pread/pwrite、向量读写 readv/writev、stat/fstat/truncate 都应该看到同一份内容和长度。MemFS 因此承担了测试期间的临时文件后端，而不仅是简单的键值映射。

### 5.3 权限、符号链接与路径解析

bounded LTP 当前覆盖了 access01、faccessat01、open02/open03、symlink\*、readlink\*、lstat\* 等 case。这些测试除了验证“文件是否存在”，还会检查父目录搜索权限、fsuid/fsgid、O\_NOFOLLOW、空路径、符号链接循环、错误码和元数据。wll0s 因此在 VFS 层统一实现更接近 Linux 的路径解析和权限检查。

权限模型围绕任务凭据和 VFS 访问检查展开：任务带有 real/effective/saved uid/gid、fsuid/fsgid 和 supplementary groups，文件访问可以按 real identity 或 filesystem identity 判断。该设计已经覆盖当前 LTP 权限用例，并为后续 capability、ACL、mount namespace 和 sticky bit 等语义扩展预留了清晰接口。

### 5.4 管道、poll 与非阻塞 I/O

管道和 poll/select 是 BusyBox、shell、lmbench 与部分 libc 测试的基础。当前 pipe 使用有界内核缓冲区，读端无数据时可阻塞或在 O\_NONBLOCK 下返回 EAGAIN，写端无读者时返回 EPIPE 并触发 SIGPIPE。ppoll 和 pselect6 通过 IO wait queue 睡眠，避免忙等。pipe 的可读可写、HUP、ERR 与非阻塞语义已经支撑当前测试面，后续可继续扩展共享 open-file description 的状态一致性。

### 5.5 ext4 与 MemFS 协作

运行时镜像中的真实程序主要位于 ext4 根目录，例如 /glibc 和 /musl。这些目录承载 BusyBox、动态库、libc-test、LTP 二进制和各类脚本。MemFS 则更适合承担运行时生成内

容，例如 `/tmp`、`/var/tmp`、测试临时文件、覆盖层和伪设备。两个后端通过 VFS 暴露统一路径接口，但持久化能力和元数据能力不同：`ext4` 更接近真实磁盘文件系统，`MemFS` 更像内核运行期的可写覆盖。

这种组合使内核既能读取评测镜像中的真实二进制，又能在测试期间创建、删除和修改临时文件。跨后端 `rename`、`link`、`symlink` 和 `metadata` 同步由 VFS 统一协调，当前实现已经覆盖常见同后端和 `overlay` 场景，后续可继续扩展跨挂载语义。

| 后端                       | 优势                                             | 后续优化方向                                                                     |
|--------------------------|------------------------------------------------|----------------------------------------------------------------------------|
| <code>ext4</code>        | 真实镜像、真实目录和文件元数据                                | 继续增强 <code>journal</code> 、 <code>dirty tracking</code> 和复杂跨目录替换。          |
| <code>MemFS</code>       | 可写、快速、适合临时对象                                   | 继续完善 <code>owner</code> 、 <code>mode</code> 持久化和 <code>metadata</code> 同步。 |
| 伪设备                      | 行为简单、兼容 <code>libc</code> 检查                   | 继续扩展 <code>tty</code> 、 <code>random</code> 等常用伪设备。                        |
| <code>Pipe/Socket</code> | 支撑 <code>IPC</code> 和 <code>poll/select</code> | 继续增强外部网络路径和 <code>socket blocking</code> 语义。                               |

## 5.6 向量 I/O 与固定偏移 I/O

`writew01` 是当前 `bounded LTP` 的入口 case 之一，它验证 `writew` 的用户 `iovec` 读取、分段长度求和、错误指针、部分写入和 `fd` 后端行为。`iozone` 又会进一步检验 `pread`、`pwrite`、`preadv`、`pwritev` 的吞吐表现。`wll0s` 因此在 `syscall/fs` 路径中保留 `IoVec` ABI 结构，并把向量 I/O 合并到普通 `fd` 读写路径上。

```
#[repr(C)]
#[derive(Clone, Copy)]
struct IoVec {
    iov_base: *mut u8,
    iov_len: usize,
}

const SMALL_IO_STACK_BUF: usize = 4096;
const VECTORED_STACK_BUF: usize = 16 * 1024;
const VECTORED_IO_CHUNK: usize = 64 * 1024;
```

代码 5-2 用户态 `iovec` ABI

这几个常量反映了向量 I/O 的内存控制策略：小 I/O 使用栈内缓冲，较大的向量读写分块处理，再交给后端 `fd` 的 `read/write` 或 `read_at/write_at`。这样既能保持 ABI 兼容，又能控制测试压力下的内核临时内存占用。

## 5.7 文件描述符生命周期

文件描述符是用户程序与内核 I/O 对象之间的统一句柄。`wll0s` 的 `fd` 表不仅保存文件对象，还保存访问模式、`offset`、`status flags`、`FD_CLOEXEC` 和资源上限。这样的设计使普通文件、目录、`pipe`、`socket`、伪设备和内存文件都能够同一套 `read/write/close/fcntl/poll` 路径下工作。

`execve`、`dup3`、`pipe2`、`socket` 和 `fcntl` 都会影响 fd 生命周期。`FD_CLOEXEC` 在 `execve` 时关闭描述符，`O_NONBLOCK` 改变 pipe/socket 的等待行为，`O_APPEND` 改变普通文件写入位置，`RLIMIT_NOFILE` 限制打开数量。对评测来说，这些状态比单次 read/write 更重要，因为 shell 和 libc 会依赖这些细节组织复杂工作流。

| 阶段   | 相关接口                                                                | 内核处理                                                                           |
|------|---------------------------------------------------------------------|--------------------------------------------------------------------------------|
| 创建   | <code>openat</code> 、 <code>pipe2</code> 、 <code>socket</code>      | 分配 fd，记录对象、访问模式和初始 flags。                                                      |
| 复制   | <code>dup</code> 、 <code>dup3</code> 、 <code>fcntl(F_DUPFD*)</code> | 复制描述符入口，并处理 <code>close-on-exec</code> 标志。                                     |
| 状态修改 | <code>fcntl(F_SETFD/F_SETFL)</code>                                 | 更新 <code>FD_CLOEXEC</code> 、 <code>O_NONBLOCK</code> 和 <code>O_APPEND</code> 。 |
| 读写   | <code>read/write/readv/writev</code>                                | 统一进入 fd 后端，维护 offset 和阻塞语义。                                                    |
| 关闭   | <code>close</code> 、 <code>execve</code>                            | 释放 fd 表项，触发 pipe/socket 端点状态变化。                                                |

## 第 6 章 系统调用与兼容模块

### 6.1 系统调用分发

系统调用模块按 Linux syscall id 分发。`os/src/syscall/mod.rs` 保存了大量 syscall 常量，并把具体实现拆到 `fs.rs`、`mm.rs`、`process.rs`、`signal.rs`、`other.rs` 和 `net.rs`。这种拆分的好处是可以让每个子模块围绕自己的核心对象演进：进程调用操作任务和地址空间，文件调用操作 fd/VFS，内存调用操作 `MemorySet`，signal 和 futex 则操作任务等待状态。

```
pub type SyscallRet = Result<usize, SysErrNo>;

pub fn syscall(syscall_id: usize, args: [usize; 6]) -> SyscallRet {
    log::debug!("[syscall] id: {}, args: {:?}", syscall_id, args);

    match syscall_id {
        SYSCALL_OPENAT => fs::sys_openat(
            args[0] as isize,
            args[1] as *const u8,
            args[2],
            args[3],
        ),
        SYSCALL_CLONE => process::sys_clone(
            args[0], args[1], args[2], args[3], args[4],
        ),
        SYSCALL_MMAP => mm::sys_mmap(
            args[0], args[1], args[2], args[3], args[4], args[5],
        ),
        _ => Err(SysErrNo::ENOSYS),
    }
}
```

代码 6-1 系统调用分发入口示意

真实代码中的 `match` 分支比这里长得多。本文只保留代表性片段，用于说明分发思路。系统调用返回值统一使用 `Result<usize, SysErrNo>`，成功路径直接返回寄存器值，失败路径由 `trap` 层转成 Linux 负 `errno`。对于尚未接入的 syscall，`ENOSYS` 能让调用者和调试日志明确识别后续实现目标。

### 6.2 用户内存访问

系统调用经常需要读取用户指针，例如路径字符串、`argv/envp/iovec/stat` 结构、`timespec/sigset`。wll0s 把用户内存复制放在 `syscall/user` 辅助路径中，并在必要时切回内核页表、再恢复当前用户页表。这样做可以避免内核直接解引用用户地址导致页表上下文混乱，也便于对 `EFAULT` 做统一处理。

用户内存访问对 LTP 很敏感。许多 LTP case 会传入坏指针、零长度、临界地址或不可写缓冲区，期望内核返回精确 `errno`。wll0s 将这类检查统一收敛到用户内存复制辅助路径中，使 `fs`、`mm`、`signal`、`time` 等多个子模块共享一致的 `EFAULT` 处理语义。

### 6.3 信号、futex 与时间接口

`signal.rs`、`other.rs` 和 `timer.rs` 支撑了当前测试中较难的兼容点。信号模块实现了 `rt_sigaction`、`rt_sigprocmask`、`rt_sigsuspend`、`rt_sigtimedwait`、`rt_sigreturn`、`kill/tkill/tgkill` 的部分语义；`futex` 路径支持 `WAIT`、`WAKE`、`WAIT_BITSET`、`WAKE_BITSET` 和 robust list 的基础清理；时间路径支持 `nanosleep`、`clock_gettime`、`gettimeofday`、`times`、`getrusage`、`setitimer/getitimer` 等。

```
pub struct TaskControlBlockInner {
    pub interval_timers: IntervalTimers,
    pub default_timer_slack_ns: usize,
    pub current_timer_slack_ns: usize,
}

// LTP setitimer/getitimer cases use three timer classes:
// ITIMER_REAL    delivers SIGALRM.
// ITIMER_VIRTUAL delivers SIGVTALRM.
// ITIMER_PROF   delivers SIGPROF.
// The path covers readback, invalid which and bad pointers.
```

代码 6-2 interval timer 的任务内状态

时间接口当前重点覆盖 `libc` 和 bounded LTP 的可观察语义：读取当前时间、执行 `sleep`、设置 interval timer、在到期时向任务投递对应信号，并对非法参数返回正确 `errno`。后续可继续扩展 `virtual/prof` CPU 时间统计和更精细的 timer slack，以支撑更复杂的定时器评测。

### 6.4 兼容性状态表

| 类别     | 完成情况 | 说明                                                                                                           |
|--------|------|--------------------------------------------------------------------------------------------------------------|
| 进程生命周期 | 已支持  | <code>clone/execve/exit/wait4/yield</code> 由真实任务状态支撑，后续可扩展作业控制和 <code>ptrace</code> 类能力。                     |
| 内存接口   | 已支持  | <code>brk</code> 、匿名 <code>mmap</code> 、 <code>mprotect/munmap</code> 、 <code>SysV shm</code> 覆盖当前用户程序和评测路径。 |
| 文件路径   | 已支持  | <code>openat/stat/readlink/symlink/access</code> 等覆盖当前 LTP，VFS 接口可继续扩展 <code>mount</code> 和权限模型。             |
| FD I/O | 已支持  | 普通文件、 <code>pipe</code> 、伪设备、 <code>socket fd</code> 、向量 I/O 和固定偏移 I/O 已接入统一 <code>fd</code> 路径。             |



| 类别 | 完成情况 | 说明                                                          |
|----|------|-------------------------------------------------------------|
| 信号 | 已支持  | action、mask、pending、sigtimedwait、sigreturn 支撑当前 libc 与评测路径。 |
| 网络 | 扩展中  | 本地 socket 队列和 fd 生命周期已接入，后续可扩展真实外部 TCP/IP 数据路径。             |

## 6.5 errno 策略

在真实评测里，返回值常常比“有没有实现”更重要。LTP 很多 case 只触发失败路径，例如坏指针应返回 `EFAULT`，无权限应返回 `EACCES` 或 `EPERM`，符号链接循环应返回 `ELOOP`，不存在路径应返回 `ENOENT`，目录类型错误应返回 `ENOTDIR` 或 `EISDIR`。如果内核为了让程序继续运行而无条件返回 0，短期可能绕过一个脚本，长期会让定位变得困难。

因此 wll0s 的兼容策略是：已覆盖路径返回真实结果；后续待接入的 syscall 返回 `ENOSYS`；参数非法返回可解释 `errno`；libc 启动和评测路径所需的兼容行为集中在对应模块中实现。该策略使系统调用语义可追踪、可测试、可持续扩展。

| 场景          | 推荐返回                       | 说明                                    |
|-------------|----------------------------|---------------------------------------|
| 待接入 syscall | <code>ENOSYS</code>        | 让调用者和日志明确识别后续实现目标。                    |
| 用户坏指针       | <code>EFAULT</code>        | 由用户内存复制辅助路径统一处理。                      |
| flag 组合非法   | <code>EINVAL</code>        | 如 clone flag 依赖关系、mmap prot/flags 冲突。 |
| 权限受限        | <code>EACCES/EPERM</code>  | 路径访问与身份切换需要区分语义。                      |
| 资源受限        | <code>ENOMEM/EMFILE</code> | 内存和 fd 上限应返回明确错误。                     |

## 6.6 跨架构兼容细节

RISC-V64 和 LoongArch64 在大多数 Linux syscall ABI 上相似，但仍存在参数顺序、寄存器、TLS 设置、入口准备和平台定时器差异。`sys_clone` 中 LoongArch64 对 TLS 和 child tid 的参数位置就需要特殊处理；trap frame 的准备也通过架构条件分支收敛到 `prepare_user_trapframe`。如果这些差异散落在各个模块中，后续双架构回归会很难维护。

```
#[cfg(target_arch = "loongarch64")]
let (tls, child_tid) = (arg4, arg3);

#[cfg(not(target_arch = "loongarch64"))]
let (tls, child_tid) = (arg3, arg4);

if (clone_bits & CLONE_SETTLS) != 0 {
    child_tf[TrapFrameArgs::TLS] = tls;
}
```

代码 6-3 clone 参数的架构差异处理

文档中保留这一点，是因为双架构支持需要同时处理编译目标、寄存器约定、trap frame 和平台计时器。同一个 syscall 在不同架构上可能有 ABI 细节差异，同一个用户程序也可能因为动态链接器或平台计时器差异表现不同。后续每个新 syscall 或新测试组，都将同步考虑 RISC-V64 与 LoongArch64 的运行路径。

## 6.7 评测导向的 ABI 覆盖

wll0s 的系统调用实现以真实用户程序运行路径为牵引。基础进程接口优先支撑 fork/exec/wait 模型，文件接口优先支撑 BusyBox、脚本执行和 benchmark I/O，内存接口优先支撑 libc 分配器、动态链接器和共享内存，信号与 futex 接口优先支撑 libc 线程与同步路径，时间接口优先支撑 sleep、poll timeout 和 interval timer。

这种覆盖方式使系统调用开发与评测证据保持一致。每个新增接口都需要明确它服务的用户程序、涉及的内核对象、成功路径和 errno 路径。对于已经接入的接口，后续优化重点放在语义精度、性能和跨架构一致性上；对于新接口，则优先补齐与当前评测集合相邻的能力。

| 类别 | 代表接口                                 | 支撑目标                              |
|----|--------------------------------------|-----------------------------------|
| 进程 | clone、execve、wait4、exit_group        | 支撑 shell、测试 runner 和父子进程模型。       |
| 内存 | brk、mmap、mprotect、shm*               | 支撑 libc 分配器、动态链接和 benchmark 共享内存。 |
| 文件 | openat、readv/writev、statx、readlinkat | 支撑 BusyBox、LTP 路径语义和文件吞吐。         |
| 同步 | futex、rt_sig*、setitimer              | 支撑 libc 同步、信号等待和 interval timer。  |
| 时间 | clock_gettime、nanosleep、ppoll        | 支撑超时、计时和 benchmark 运行控制。          |

## 第 7 章 LTP 与评测模块

### 7.1 harness 设计

harness 是 `feat/ltp` 分支区别于普通教学内核的关键模块。它扫描镜像中的 `glibc/musl` 根目录，按测试组运行真实脚本或二进制，并在串口中输出 `judge` 能识别的分组标记。对于 bounded LTP，harness 还支持通过编译期环境变量 `LTP_CASES` 启动聚焦 collector，运行指定 case 集合。没有 `LTP_CASES` 时，可以回落到真实 LTP 脚本路径。

```
fn ltp_cases_filter() -> Option<'static str> {
    match option_env!("LTP_CASES") {
        Some(filter) => {
            let filter = filter.trim();
            if filter.is_empty() || filter.eq_ignore_ascii_case("all") {
                None
            } else {
                Some(filter)
            }
        }
        None => None,
    }
}

fn run_ltp_collection_harness(script_path: &str, cases_filter: &str) -> bool {
    let Some((root, _logical_script)) = logical_path_for_script(script_path) else {
        return false;
    };
    let group_name = ltp_group_name(&root);
    console_write("#### OS COMP TEST GROUP START ");
    console_write(group_name);
    console_write(" ####\n");

    for part in cases_filter.split(',') {
        let case = part.trim();
        if !case.is_empty() {
            run_ltp_case(&root, case);
        }
    }

    console_write("#### OS COMP TEST GROUP END ");
    console_write(group_name);
    console_write(" ####\n");
    true
}
```

代码 7-1 LTP case collector

这段代码显示出一个重要原则：bounded LTP 通过 `/ltp/testcases/bin/<case>` 的真实二进制输出形成评测结果。collector 负责选择 case 和包装分组输出，每个 case 的通过或失败由测试程序输出，最后由 judge 脚本解析。该路径保证了评测结果来自真实用户态执行。

## 7.2 测试集合

当前分支的测试体系覆盖多个层次。基础测试验证 fork、clone、wait、pipe、brk、mmap 等基础能力；BusyBox 验证 shell、路径、文件和脚本执行；Lua 验证解释器和 libc 运行时；iozone 验证文件 I/O、共享内存和多进程吞吐；libc-test 验证 libc ABI 广度；libcbench 和 lmbench 验证性能压力；bounded LTP 则验证更细的 Linux errno 和接口语义。

| 测试组       | 目标      | 主要压力点                                          |
|-----------|---------|------------------------------------------------|
| basic     | 基础功能    | 进程、管道、等待、堆、匿名映射、基本文件。                          |
| busybox   | 用户态工具链  | shell、脚本、路径解析、目录项、poll/select。                 |
| lua       | 解释器运行   | 动态链接、文件 I/O、时间、随机数、futex。                      |
| iozone    | 文件吞吐    | ext4 写缓存、pread/pwrite、pwritev/preadv、SysV shm。 |
| libc-test | libc 兼容 | 信号、线程、内存映射、errno、动态链接。                         |
| lmbench   | 系统性能    | fork/exec、pipe、上下文切换、文件和时间接口。                  |
| LTP       | 接口语义    | 身份、权限、符号链接、readlink/lstat、interval timer。      |

## 7.3 有界 LTP 选择

当前 FOCUSED\_LTP\_CASES 的选择集中在四类能力。第一类是向量写和 fd 路径，例如 writev01；第二类是身份与组，例如 setegid02、getgroups01、setgroups01 到 setgroups04；第三类是文件权限与符号链接，例如 access01、faccessat01、open02/open03、symlink\*、readlink\*、lstat\*；第四类是 interval timer，例如 setitimer01/02 和 getitimer01/02。这些 case 有共同特点：它们能明确验证一个子系统的真实语义，同时让评测依赖面保持清晰可控。

后续扩展 LTP 时，将按“一个模块，一个主题，一组 case”的方式推进。例如扩展文件权限时，先强化 VFS 权限和 errno，再加入更多 open/access 组合；扩展时间时，先补充 virtual/prof CPU 时间，再加入更复杂的 timer case；扩展信号时，先完善 pending queue 和 ucontext，再加入实时信号或 sigaltstack 类测试。这样文档、代码和测试证据能够保持同步。

## 7.4 parser 分组协议

评测脚本依赖串口日志中的分组协议。harness 输出 OS COMP TEST GROUP START 和 OS COMP TEST GROUP END，judge 脚本再根据组名选择对应 parser。这个协议决定了测试

结果是否能被评测系统稳定识别，因此组名、END 标记、用户程序输出和内核日志需要保持清晰分离。

```
#### OS COMP TEST GROUP START ltp-glibc ####
RUN LTP CASE access01
...
#### OS COMP TEST GROUP END ltp-glibc ####

#### OS COMP TEST GROUP START ltp-musl ####
RUN LTP CASE access01
...
#### OS COMP TEST GROUP END ltp-musl ####
```

代码 7-2 串口分组输出格式

因此内核日志要清晰区分调试信息和用户输出。开发时可以打开更多 `syscall log`，提交或验收时则保证 `judge parser` 看到稳定的 `START`、`END`、`case` 输出和结果行。LTP 结果由真实测试程序输出，内核负责提供运行环境和输出通道。

## 7.5 评测证据闭环

bounded LTP 的评测闭环由四个环节组成：构建阶段记录内核镜像和 `feature` 配置，运行阶段保存 QEMU 串口日志，解析阶段由 `judge` 脚本提取测试结果，归档阶段保存命令、提交号、镜像哈希和 `summary`。这个闭环使每次结果都可以追溯到具体二进制和具体命令，也便于后续回归对比。

对比赛评审而言，可复现证据比单次截图更有说服力。wllos 的 runner 以 `--suite`、`--arch`、`--runs` 和 `LTP_CASES` 控制运行范围，串口日志保留用户程序输出，`summary` 文件保留结构化结果。文档中的测试结论均以这条证据链为基础。

| 环节 | 产物                                                                       | 作用                                   |
|----|--------------------------------------------------------------------------|--------------------------------------|
| 构建 | <code>kernel-rv</code> 、 <code>kernel-la</code> 、 <code>build log</code> | 记录架构、 <code>feature</code> 和最终内核二进制。 |
| 运行 | <code>serial.log</code> 、QEMU exit code                                  | 保存真实用户程序输出和内核日志。                     |
| 解析 | <code>judge-summary.json</code> 、 <code>parser</code> 输出                 | 将串口文本转为测试组结果。                        |
| 归档 | <code>git-rev.txt</code> 、 <code>command.txt</code> 、 <code>hash</code>  | 保证后续复跑和结果追溯。                         |

## 7.6 新增 case 准入标准

新增 LTP case 前，应先回答三个问题：它验证哪个内核对象；失败时最小复现是什么；通过后会破坏已有测试组。一个 case 可能表面属于文件系统，但实际依赖用户指针、凭据、路径解析、VFS 后端和 `signal` 中断。把依赖拆清楚后再实现，能减少“修一个 case 坏三个 group”的情况。

| 阶段      | 动作                                  | 验收点                        |
|---------|-------------------------------------|----------------------------|
| 选择 case | 只选一个相邻子系统                           | 能说明目标 syscall、errno 和依赖对象。 |
| 最小运行    | --suite ltp 加 单 个<br>LTP_CASES      | 串口日志中能看到真实二进制输出。           |
| 实现修复    | 补真实语义                               | 成功路径和失败路径都有明确 errno。       |
| 回归验证    | 跑相关 basic/busybox/lua/<br>benchmark | 新增 case 不破坏已有默认路径。         |
| 纳入集合    | 更新 FOCUSED_LTP_CASES<br>和文档         | case 名、证据和能力描述保持同步。        |

## 第 8 章 测试结果与分析

### 8.1 编译与运行方式

Typst 文档生成与内核测试是两条不同路径。本文档使用 `typst compile` 生成 PDF；内核本身使用 `make` 和 QEMU 运行。`feat/ltip` 分支提供了多种脚本，既可以跑默认 `all profile`，也可以只跑 `focused LTP`。为了便于复现，文档列出最小命令，不要求读者一次性运行所有慢速评测。

```
# build this document
typst compile typst-doc/wllos-ltp.typ typst-doc/build/wllos-ltp.pdf

# print focused LTP cases
make -C ltp print-ltp-cases

# build RISC-V64 kernel with LTP feature
make -C ltp ARCH=riscv64 build LTP=1

# run focused LTP and keep serial logs for judge parser
python ltp/scripts/perf_baseline_runner.py --arch riscv64 --suite ltp --runs 1
```

代码 8-1 文档与内核的常用命令

评测运行依赖镜像、QEMU、工具链和运行时间，因此文档采用可复现实验记录方式保存结论。每次关键改动后保存串口日志、`runner summary` 和 `judge` 解析结果，并在 README 或开发日志中记录 `commit`、架构、`suite`、命令和结果。这样一旦出现回归，能够快速定位是内核语义、构建配置、镜像内容还是 `parser` 变化。

### 8.2 当前能力总结

结合仓库中的 `syscall matrix` 和评测脚本，当前分支已经具备以下可观察能力：基础进程生命周期可运行，匿名内存和 `heap` 可支撑 `libc`，`ext4/MemFS/fd/pipe` 能支撑 `BusyBox` 和文件测试，动态 ELF 与解释器路径能启动 `glibc/musl` 用户程序，`bounded LTP case` 有真实二进制路径，`libc-test`、`iozone`、`libcbench`、`lmbench` 都有对应 `harness` 入口和 `judge` 脚本。

| 能力 | 已具备内容                                      | 后续优化方向                                                                                                                                      |
|----|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 任务 | <code>clone/execve/wait4/exit_group</code> | 已支撑进程生命周期，后续可扩展 <code>pthread</code> 、作业控制、 <code>ptrace</code> 和进程组等待。                                                                     |
| 内存 | <code>brk/mmap/munmap/mprotect/shm</code>  | 已支撑 <code>libc</code> 和 <code>benchmark</code> 路径，后续可扩展 <code>dirty tracking</code> 、 <code>page cache</code> 和精确 <code>accounting</code> 。 |

| 能力 | 已具备内容                                 | 后续优化方向                                                     |
|----|---------------------------------------|------------------------------------------------------------|
| 文件 | open/read/write/stat/symlink/readlink | 已覆盖核心路径，后续可扩展跨后端 rename/link、权限和持久化语义。                     |
| 信号 | sigaction/sigprocmask/sigtimedwait    | 已支撑当前信号路径，后续可扩展实时信号队列、ucontext 和 syscall restart。          |
| 时间 | clock_gettime/nanosleep/setitimer     | 已支撑时间与 interval timer，后续可扩展 virtual/prof timer 的 CPU 时间统计。 |
| 网络 | 本地 socket 队列                          | 已接入 socket fd 生命周期，后续可扩展外部网络栈和性能评测。                        |

### 8.3 开发问题与解决方法

开发过程中，wll0s 将问题定位从“观察现象”推进到“定位对象、复现路径、沉淀方案”。一个用户态异常通常会跨越多个模块：LTP access case 会同时涉及路径解析、父目录搜索权限、fsuid/fsgid、VFS metadata 和用户指针复制；BusyBox 脚本执行会同时涉及 shebang、execve 参数栈、动态解释器、/proc/self/exe 和 fd close-on-exec；iozone 会同时放大 ext4 写缓存、SysV shm、pwritev、fork 后 fd offset 和 wait4 语义。wll0s 的处理方式是先用串口日志确认最后一个用户程序，再用 focused case 缩小范围，最后把解决方法沉淀回模块接口。

| 现象            | 定位方法                                             | 解决方法与沉淀                                                                |
|---------------|--------------------------------------------------|------------------------------------------------------------------------|
| 动态程序启动异常      | execve、ELF loader、用户栈、auxv、解释器                   | 统一构造 argc/argv/envp/auxv，补齐解释器装载、signal trampoline 和 close-on-exec 处理。 |
| 路径 errno 不一致  | VFS resolver、父目录权限、符号链接、credentials              | 在 VFS 层集中处理 fsuid/fsgid、搜索权限、O_NOFOLLOW 和符号链接深度，返回可解释 errno。           |
| 等待或超时停滞       | WaitQueue token、timer deadline、signal、child exit | 入队后重检条件，使用 token 区分过期唤醒，并在任务退出时清理 wait/futex/timer 队列。                 |
| 大文件与向量 I/O 压力 | iozone/lmbench、fd offset、preadv/pwritev、sync     | MemFS 使用 chunked/sparse 表示，向量 I/O 分块复制，并把固定偏移读写收敛到 fd 后端。              |
| 单 case 回归     | LTP_CASES=case、串口日志、judge summary                | 按“一个模块、一个主题、一组 case”推进，保存命令、commit、日志和 parser 结果用于回归。                  |



## 8.4 回归证据记录格式

为了让测试结果可复现，建议每次关键运行都记录同一组字段。只写“通过”没有太大价值，因为无法区分架构、镜像、命令、case 集和 parser 是否一致。比较可靠的记录方式应包含 commit、架构、构建 feature、运行命令、QEMU 退出码、串口日志路径、parser summary 和失败 case 列表。

| 字段   | 示例说明                                          |
|------|-----------------------------------------------|
| 提交   | feat/ltp 分支具体 commit，例如本地 checkout 的短 hash。   |
| 架构   | riscv64 或 loongarch64，不要混写。                   |
| 构建选项 | LTP=1、LTP_CASES=...、IOZONE=1、LMBENCH=1 等。     |
| 运行命令 | 完整 runner 命令，包含 suite、runs 和是否删除 sdcard copy。 |
| 日志路径 | 串口日志、runner summary、judge 输出的保存位置。            |
| 结论   | 通过数量、失败数量、失败 case、首个异常 syscall 或 panic。       |

```
commit: 583680e
arch: riscv64
suite: ltp
case: access01
command: python scripts/perf_baseline_runner.py --arch riscv64 --suite ltp --runs
1
kernel: LTP=1 LTP_CASES=access01
evidence:
- qemu exit code
- serial log path
- judge_ltp summary
- first failing syscall if any
```

代码 8-2 推荐的单 case 调试记录

这类记录可以直接贴进开发日志，也可以后续转为自动生成表格。它的价值在于把“测试通过”变成可检查证据：当未来某个模块改动导致回归时，开发者可以用同一命令、同一 case 和同一 parser 复跑，而不需要重新猜测当初的环境。

## 第 9 章 总结与展望

### 9.1 总结

wlllos `feat/ltp` 分支已经从普通教学内核推进到面向真实评测的内核形态。它围绕真实用户程序建立了一组可运行、可解释、可回归的核心语义：地址空间以 `MemorySet` 和 `VMA` 为中心，任务以 `TaskControlBlock` 和 `thread group` 为中心，文件系统以 `fd/VFS/backing object` 为中心，系统调用以 `Linux id` 分发和 `errno` 为中心，`harness` 以真实测试程序输出为中心。

与 `UndefinedOS` 文档相比，本文档保留了正式技术文档的排版结构：封面、自动目录、页眉页脚、章标题、节标题、表格、图注、灰底代码块和代码编号；内容上则更贴近 wlllos 的当前分支，把旧稿中分散的章节合并进模块内部。这样章节数量更少，阅读路径更清楚，但每个模块仍然有足够的设计说明和关键代码，后续可以继续沿着模块扩写。

### 9.2 后续工作

后续工作建议按优先级推进。第一，继续强化 `VFS` 和 `fd` 语义，尤其是共享 `open-file description`、跨后端 `rename/link`、权限检查和持久化能力。第二，完善线程和 `futex`，扩展 `pthread` 常见路径、`robust futex`、共享 `futex key` 和取消语义。第三，改进 `signal` 和 `timer`，包括 `ucontext`、实时信号队列、`syscall restart` 和 `virtual/prof CPU` 时间。第四，在文件 I/O 上增强 `dirty tracking`、`msync`、稀疏文件和更高效的 `pwritev/preadv`。第五，在网络方面推进 `virtio-net` 或最小 `TCP/IP` 数据路径。

LTP 扩展将继续保持按模块推进的节奏。每增加一组 `case`，都明确它验证的内核对象、需要的 `errno`、涉及的用户指针和可能影响的已有测试。当真实二进制、串口日志、`judge parser` 和回归 `suite` 都给出稳定证据后，再将该 `case` 纳入默认集合。这样的节奏有利于保持竞赛内核的可维护性和可复现性。

## 第 10 章 AI 辅助使用说明

### 10.1 使用原则

wlllos 开发小组在项目推进过程中健康、审慎地使用 AI 工具。AI 的角色是辅助理解、辅助整理和辅助检查，而不是替代核心设计、代码实现和测试验证。涉及内核语义、模块接口、评测结论和性能判断的内容，均以队员阅读源码、运行程序、分析日志和调试复现后的结果为准。

这种使用方式提高了开发效率，也保持了工程判断的独立性。队员会主动识别 AI 输出中可能存在的错误、幻觉、不完整推理和与当前代码不一致的内容；对于无法由仓库代码、运行日志或测试结果确认的说法，不写入技术结论，也不作为实现依据。

### 10.2 使用场景与验证方式

| AI 辅助场景  | 具体用途                                                      | 队员验证方式                                |
|----------|-----------------------------------------------------------|---------------------------------------|
| 规范理解     | 辅助阅读 Linux ABI、系统调用语义、errno 行为和 LTP case 意图。              | 队员结合官方语义、测试源码、内核实现和实际运行结果自行确认。        |
| 测试思路生成   | 帮助整理边界条件、异常参数、回归路径和 focused case 选择。                      | 队员将测试思路转化为可运行命令、串口日志和 parser summary。 |
| 代码审查     | 提示资源生命周期、锁粒度、用户指针、错误码和跨模块协作等关注点。                          | 队员逐项判断建议是否适合当前代码结构，采纳前阅读相关实现。         |
| bug 原因分析 | 根据日志和现象列出可能原因，例如路径解析、COW、wait queue、fd offset 或 timer 语义。 | 队员通过单 case 复现、日志对比、断点或最小修改验证真实原因。     |
| 文档表达     | 帮助组织章节、压缩冗余描述、统一技术报告语气和排版说明。                              | 队员核对文档与源码一致，不虚构技术实现、测试结果和支持范围。        |
| 输出质量控制   | 辅助发现表述含混、证据不足或与上下文不一致的内容。                                 | 队员识别并剔除错误、幻觉和不完整之处，保留可验证结论。           |

### 10.3 工程边界与责任

AI 输出不直接等同于项目事实。wlllos 的设计选择来自开发小组对代码结构、运行目标和比赛评测要求的综合判断；每一项进入文档的技术实现都应能在源码、构建脚本、测

试日志或可复现实验中找到依据。AI 可以帮助提出候选方案，但最终方案是否采纳、如何落地、是否纳入文档和评测结论，均由开发小组负责。

在本报告中，AI 被用于改善说明质量和审阅表达一致性。报告不使用 AI 虚构模块能力，不编造通过结果，不以未经验证的推测替代测试证据。对于后续开发，团队将继续保持“AI 辅助、人工判断、实测确认、证据归档”的工作方式，使工具效率服务于真实工程质量。